

前后端的搭建

🔧 个人基础

实战过较为完整的后端及客户端开发，比较熟练使用 Kotlin 及相关技术栈构建原生 Android 客户端、KMP 跨平台客户端，可快速搭建项目。

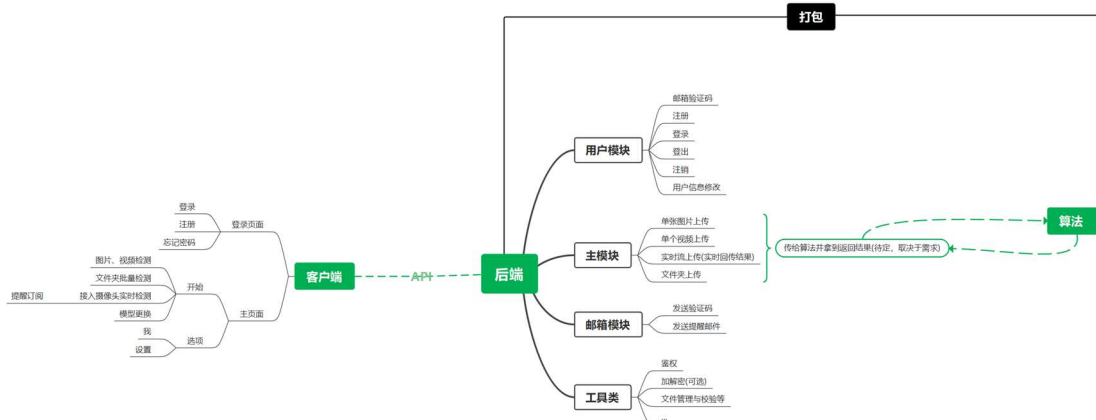
最好的客户端方案应该就是用 C#语言原生构建，但是由于经验不足，只能退而求其次使用比较熟练的 KMP 跨平台；

关于实时摄像头检测相关功能，可能还需要进一步实战学习

📋 计划

等待算法部分完成后，规划数据的组装，搭建后端，所有 API 测试通过后，开始前端/客户端的开发，完成支持上传图片、视频的 Demo 后进行成果展示，后续继续客户端功能的完善，在算法和后端不会出现大规模变更或重构时，进行打包部署到生产环境

💡 设计导图



🔥 技术栈分析

🚀 后端

整体使用 Kotlin 搭建 **SpringBoot** 单体项目，集成以下：

消息队列（Message Queue）**RabbitMQ**，用于处理并发请求

非关系型数据库（NoSQL）**Redis**，用于缓存数据

数据库 (SQL) **MySQL**, 使用 **MyBatis** 连接数据库, 用于保存用户信息

实时流前后端通信使用 **WebSocket**, 用于前端摄像头通信

Junit 用于测试

需要一个邮箱账号, 使用 SMTP 用于发送验证码和暴力检测预警

SpringBoot Actuator 用于监控

(如有进阶的安全需求, 可选择性考虑使用 AES 或 RSA 等加密请求响应)

(如有进阶的鉴权需求, 登录方案可采取 SSO, JWT 鉴权或 **Spring Security**)

(🤖 可能用到的备用技术栈: GraphQL、ElasticSearch、Kafka、MongoDB)

API (接口)

前后端分离设计, API 设计规范: **Restful API**

为方便设计, 全部统一为 JSON 数据, 基本结构都为

```
{  
  
  "state": 200,  
  
  "msg": "成功",  
  
  "data": ...  
}
```

state 使用响应码, 前后端制定好规范

msg 用于承载提示消息

data 为给前端的数据

部署

打包 **Docker**

前端/客户端

架构 MVVM  第三方开源依赖用超链接标注

Kotlin Multiplatform 写共用业务逻辑, 针对 Windows 端可能会使用一些特定的库例

如 OpenCV 等;

Compose Multiplatform 用于 UI 的绘制: Material Design 3

协程、Flow 发送与接收异步网络请求

ViewModel 界面层与逻辑层之间的“桥梁”

Ktor 跨平台的网络框架

Kotlin Serialization 反序列化 JSON, 用于解析数据

[Settings](#) 用于持久化存储

[Coil](#) 网络图片加载库

功能设计: 如首页图

⚠ 注意: 客户端的设计需要良好的健壮性, 客户端作为用户交互的直接层, 尽量避免恶性崩溃 Bug 给用户带来不好的印象 (例如: 空指针、数组越界、意外的数据-实体类与后端的数据不匹配 等), 尽量使用 try catch 包裹数据处理语句等